

Introduction to R

Aravind Hebbali

2026-10-02

Contents

Introduction to R	4
Preface	4
Structure of the book	5
Software information	5
About the Author	6
Introduction to R	7
Overview	7
Organizations	7
R Foundation	7
R Consortium	7
Installation	7
CRAN	7
Extensions	7
GUI	8
R GUI	8
RStudio	8
Install R & RStudio	9
R	9
RStudio	9
Variables in R	10
Introduction	10
What is a variable?	10
Creating Variables	10
Using Variables	10
Components of a Variable	11
Naming Conventions	11
Data Types in R	11
Introduction	11
Numeric	12
Integer	12
Character	13

Logical	14
Date	16
Getting Help in R	16
Introduction	16
Online Resources	16
R Bloggers	16
Stack Overflow	17
Twitter	17
RStudio Community	17
RStudio	17
Reddit	17
R Weekly	17
R User Groups	17
Data Helpers	17
Internal	17
help.start	18
help	18
help.search	18
demo	18
example	18
Package Documentation	18
library	18
vignette	19
browseVignettes	19
RsiteSearch	19
Vectors in R	19
Introduction	19
Numeric Vector	19
Integer Vector	20
Character Vector	21
Logical Vector	21
Naming Vector Elements	22
Method 1: Create vector and add names later	22
Method 2: Specify names while creating vector	22
Vector Coercion	22
Case 1: Different Data Types	22
Case 2: Numeric, Integer and Logical	23
Case 3: Integer and Logical	23
Vector Operations	23
Case 1: Vectors of same length	23
Case 2: Vectors of different length	24
Missing Data	24
Detect missing data	24

Omit missing data	25
Exclude missing data	25
Index Vectors	25
Out of range index	26
Negative Index	26
Subset Multiple Elements	27
Subset Named Vectors	28
Subset using logical values	28
Subset using logical expressions	29
Logical Operators	30
Dataframes in R	30
Introduction	30
Create dataframes	31
Basic Information	31
Select Columns	31
Single Column	31
Multiple Columns	32
Select Rows	32
Factors in R	33
Introduction	33
Create Factors	33
Order Levels	33
Specify Labels	34
Check Levels	34
Lists in R	34
Introduction	34
Creating Lists	34
Access List Elements	35
Coerce other objects	36
Coerce list to other objects	38
Install & Update R Packages	38
Introduction	38
Install Packages	39
CRAN	39
GitHub	39
BitBucket	39
Bioconductor	39
rForge	39
Install Different Versions	39
Installed Packages	40
Library Paths	40
Install	40
Load	40

Update Packages	40
Remove Packages	40
RStudio Projects & Data Import	40
Why avoid <code>setwd()</code>	41
Portable paths with <code>here</code>	41
Reading CSV files	41
Reading Excel files	41
Practice Check	42
Your First Visualization	43
Base R graphics	43
ggplot2 graphics	43
Practice Check	44
Your First Data Wrangling	45
The five verbs	45
Practice Check	46
Where to go next	47

Introduction to R

Preface



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

Structure of the book

Chapter Introduction to R gives a quick overview of R and its evolution over the last 2 decades. Chapter Install R & RStudio acts as a guide for installing R and RStudio. Chapters Variables in R and Data Types in R introduce the reader to variables and data types in R. Chapter Getting Help in R shows the right place and right way to ask for help. Chapters Vectors in R to Lists in R introduce the basic data structures in R. Chapter Install & Update R Packages explores the R package system.

Software information

The R session information when compiling this book is shown below:

```

sessionInfo()

R version 4.5.2 (2025-10-31 ucrt)
Platform: x86_64-w64-mingw32/x64
Running under: Windows 10 x64 (build 19045)

Matrix products: default
  LAPACK version 3.12.1

locale:
[1] LC_COLLATE=English_India.utf8  LC_CTYPE=English_India.utf8
[3] LC_MONETARY=English_India.utf8 LC_NUMERIC=C
[5] LC_TIME=English_India.utf8

time zone: Asia/Calcutta
tzcode source: internal

attached base packages:
[1] stats      graphics  grDevices datasets  utils      methods    base

loaded via a namespace (and not attached):
[1] compiler_4.5.2 fastmap_1.2.0 cli_3.6.6      htmltools_0.5.9
[5] tools_4.5.2    yaml_2.3.12  rmarkdown_2.32 knitr_1.52
[9] jsonlite_2.0.0 xfun_0.61    digest_0.6.39 rlang_1.3.0
[13] renv_1.2.4     evaluate_1.0.5

```

We do not add prompts (> and +) to R source code in this book, and we comment out the text output with two hashes ## by default, as you can see from the R session information above. This is for your convenience when you want to copy and run the code (the text output will be ignored since it is commented out). Package names are in bold text (e.g., **rmarkdown**), and function names are followed by parentheses (e.g., `mean()`). The double-colon operator `::` means accessing an object from a package.

About the Author

Aravind Hebbali is the founder of Rsquared Academy. He earned his Masters in Economics from Madras School of Economics. As an active R user, he has authored several R packages such as

- `olsrr`
- `rfm`
- `descriptr`
- `blorr`
- `xplorerr`

In 2015, he founded Rsquared Academy, a free and open source education initiative with focus on data science and analytics. Apart from self paced online courses, Rsquared Academy offers customized learning modules for corporates and universities.

You can find him on GitHub.

Introduction to R

Overview

R is a free and open source programming language widely used in academia, industry and government organisations for data science and machine learning. R is well suited for interactive data analysis, visualization and modeling. To view a list of organizations using R, [click here](#).

Before we talk about R, let us learn a bit about the S language, which was the predecessor to R. It was developed by Richard Becker, John Chambers and Allan Wilks at the Bell Laboratories. R is a modern implementation of S.

R itself was written by Ross Ihaka and Robert Gentleman at the University of Auckland. The development of R was started in 1992 and an initial version was released in 1995. Version 1.0 was released in February 2000. Version 2.0 was released in October 2004. R is under active development by the R Core Team, with regular annual major/minor releases (such as R 4.4 and R 4.5).

R is released under GNU GPL license and is currently one of the most popular tools for data science and machine learning. You can find more details and information related to R at the official website of the R project. You can download R, learn more about the R foundation, its members, activities, get the latest news about R and also donate to the R foundation.

Organizations

R Foundation

R foundation is a not for profit organization founded by the members of the R core team and is responsible for the continued development of R. It also supports the annual useR conferences. You can learn more about the R foundation [here](#).

R Consortium

R consortium provides support to the R foundation and other key organizations and groups developing, maintaining and using R. It funds projects that improve the infrastructure around R and provides support to conferences and user groups around the world. You can learn more about R consortium [here](#).

Installation

R is available for Windows, Mac and is part of many Linux distributions. You can download and install R from any of the CRAN mirrors [listed here](#).

CRAN

CRAN stands for **C**omprehensive **R** Archive **N**etwork and you can download R and user contributed packages from [here](#) for different operating systems. CRAN is pronounced as KRAN or C-RAN.

Extensions

User contributed packages can be downloaded from

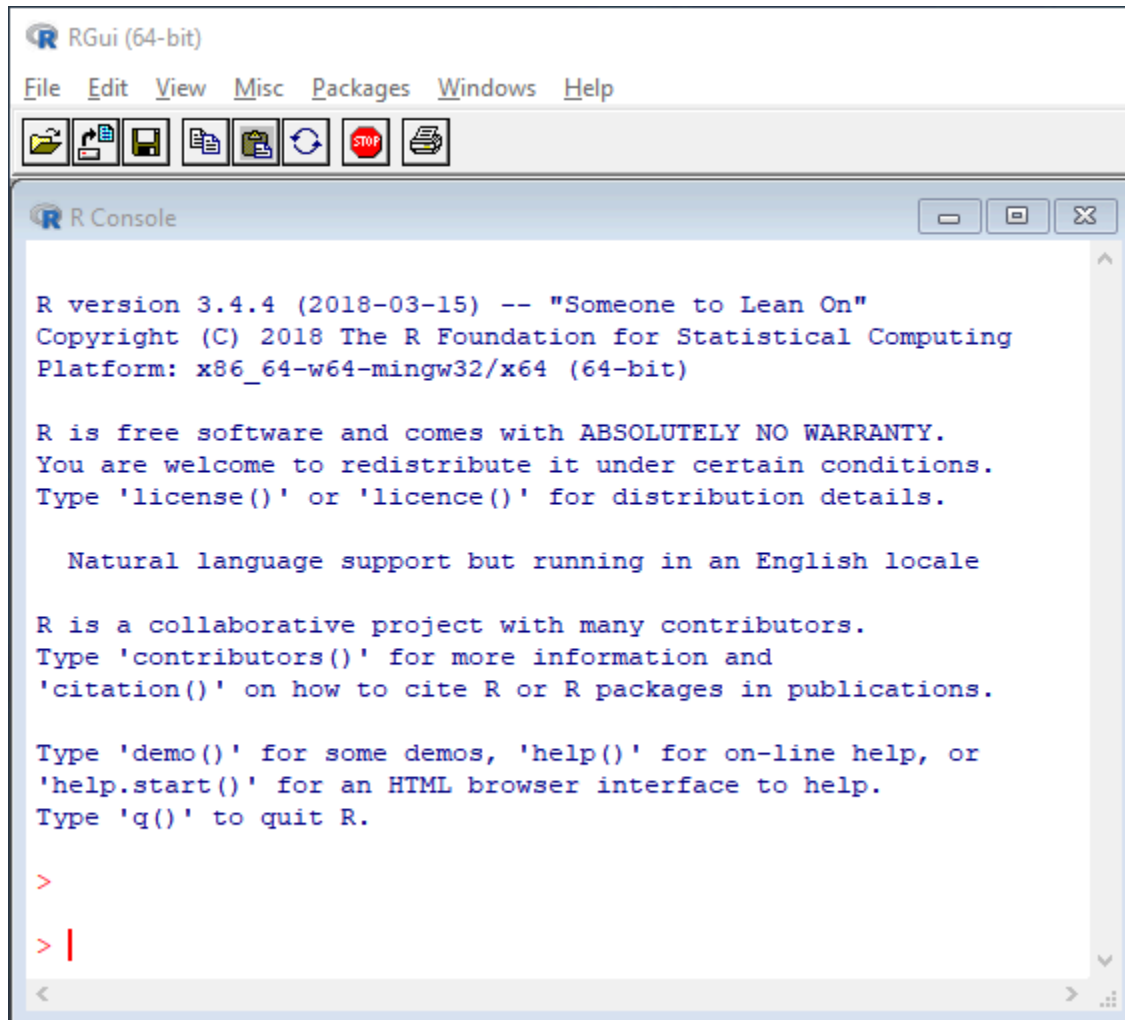
- CRAN

- Bioconductor
- GitHub
- R-Forge

GUI

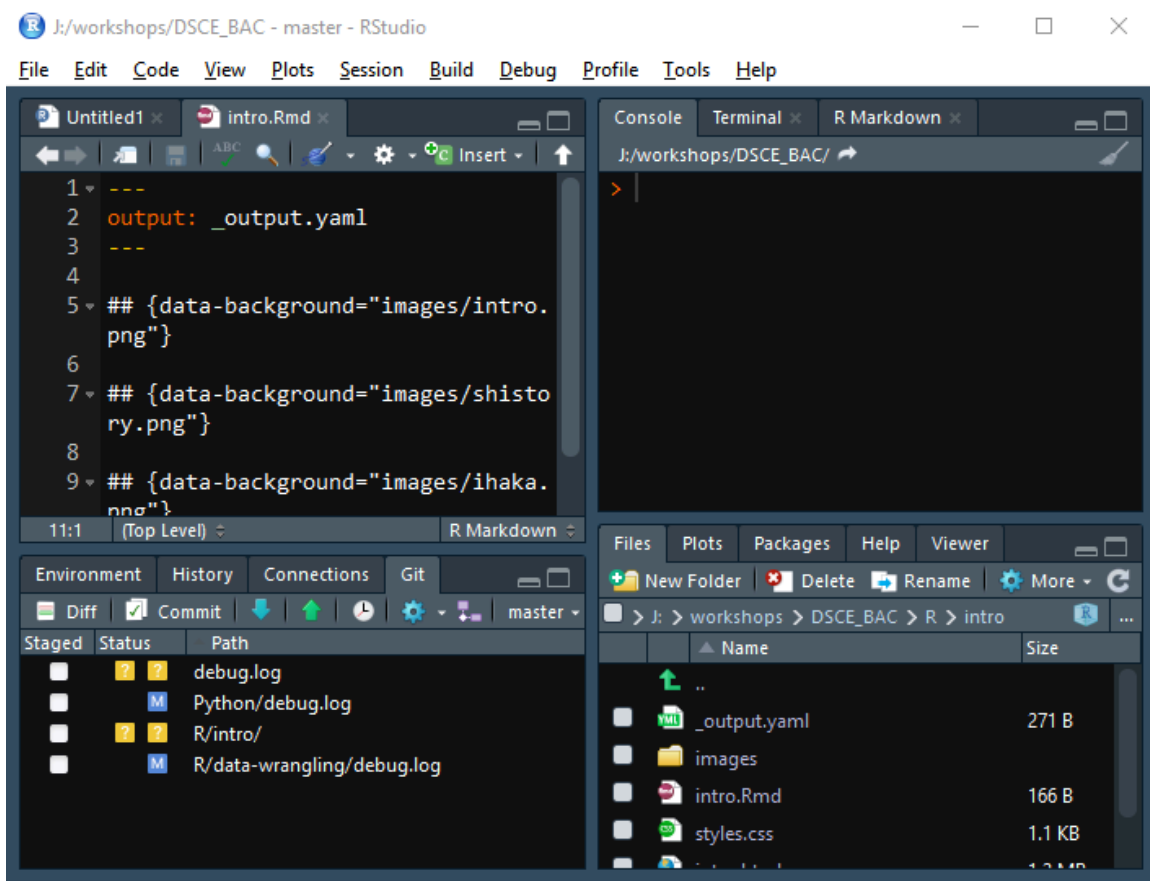
R GUI

R comes with a simple GUI with sufficient features for data analysis and visualization.



RStudio

RStudio is a very popular IDE (integrated development environment) built to suit the data science work flow. RStudio is developed by Posit (formerly RStudio, rebranded in 2022); Positron is a modern alternative IDE for R and Python.



Install R & RStudio

R

If you prefer written instructions over video, follow these steps:

- Download R for your operating system from a CRAN mirror (Windows, macOS, or Linux installers are listed there).
- Run the installer with default options.
- Verify the installation by opening a terminal and running `R.version.string`.

RStudio

If you prefer written instructions over video, follow these steps:

- Download RStudio Desktop from posit.co/download/rstudio-desktop.
- Install it after installing R (RStudio needs R to work).
- Open RStudio and create a new project via *File > New Project* (this creates an `.Rproj` file that keeps your working directory, history, and settings together).
- Verify your setup by running `R.version.string` in the R console.

Variables in R

Introduction

In the previous chapter, we learnt to install RStudio. In this chapter, we will learn about variables and data types. You can skip this chapter, if you have prior experience in any other programming language.

What is a variable?

- variables are the fundamental elements of any programming language
- they are used to represent values that are likely to change
- they reference memory locations that store information/data

Let us use a simple case study to understand variables. Suppose you are computing the area of a circle whose radius is 3. In R, you can do this straight away as shown below:

```
3.14 * 3 * 3
```

```
[1] 28.26
```

But you cannot reuse the radius or the area computed in any other analysis or computation. Let us see how variables can change the above scenario and help us in reusing values and computations.

Creating Variables

A variable consists of 3 components:

- variable name
- assignment operator
- variable value

We can store the value of the radius by creating a variable and assigning it the value. In this case, we create a variable called `radius` and assign it the value 3 using the assignment operator `<-`.

```
radius <- 3  
radius
```

```
[1] 3
```

Now that we have learnt to create variables, let us see how we can use them for other computations. For our case study, we will use the `radius` variable to compute the area of a circle.

Using Variables

We will create two variables, `radius` and `pi`, and use them to compute the area of a circle and store it in another variable `area`.

```
# assign value 3 to variable radius  
radius <- 3
```

```
# assign value 3.14 to variable pi  
pi <- 3.14
```

```
# compute area of circle
area <- pi * radius * radius

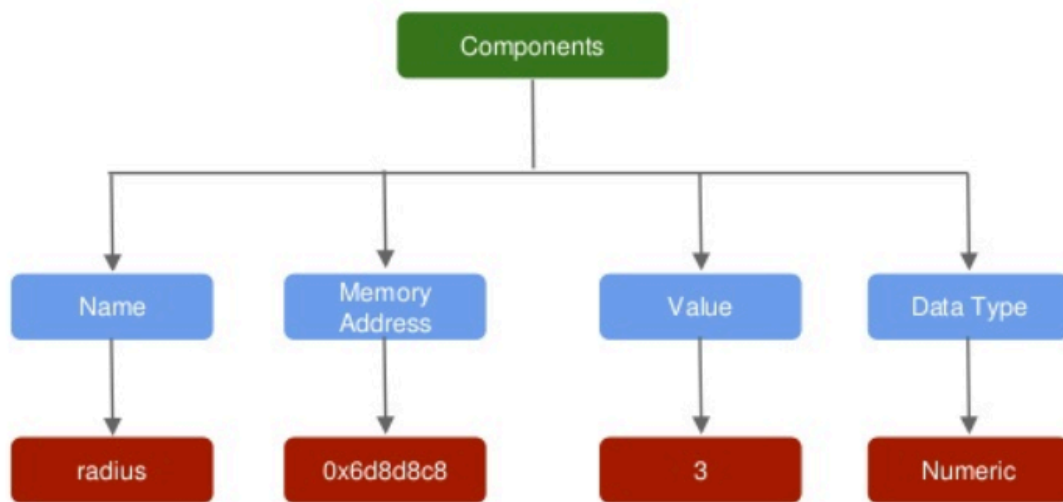
# call radius
radius

[1] 3

# call area
area

[1] 28.26
```

Components of a Variable



Naming Conventions

- Name must begin with a letter or a dot (.); e.g. `.hidden <- 42` is valid in base R. Do not use numbers, dollar sign (\$) or underscore (_) as the first character.
- The name can contain numbers or underscore. Do not use dash (-). Periods (.) are syntactically valid in R variable names, but are discouraged in modern style guides (tidyverse style prefers `snake_case`).
- Do not use the names of keywords and avoid using the names of built in functions.
- Variables are case sensitive; `average` and `Average` would be different variables.
- Use names that are descriptive. Generally, variable names should be nouns.
- If the name is made of more than one word, use underscore to separate the words.

Data Types in R

Introduction

In this chapter, we will learn about the following data types:

- numeric/double
- integer
- character
- logical
- date/time

Numeric

In R, numbers are represented by the data type `numeric`. We will first create a variable and assign it a value. Next we will learn a few methods of checking the type of the variable.

```
# create two variables
number1 <- 3.5
number2 <- 3

# check data type
class(number1)

[1] "numeric"

class(number2)

[1] "numeric"

# check if data type is numeric
is.numeric(number1)

[1] TRUE

is.numeric(number2)

[1] TRUE
```

If you carefully observe, integers are also treated as `numeric/double`. We will learn to create integers in a while. In the meanwhile, we have introduced two new functions in the above example:

- `class()`: returns the class or type
- `is.numeric()`: tests whether the variable is of type `numeric`

Integer

Unless specified otherwise, integers are treated as `numeric` or `double`. In this section, we will learn to create variables of the type `integer` and to convert other data types to `integer`.

- create a variable `number1` and assign it the value 3
- check the data type of `number1` using `class`
- create a second variable `number2` using `as.integer` and assign it the value 3
- check the data type of `number2` using `class`
- finally use `is.integer` to check the data type of both `number1` and `number2`

```
# create a variable and assign it an integer value
number1 <- 3
```

```
# create another variable using as.integer
number2 <- as.integer(3)
```

```
# check the data type
class(number1)
```

```
[1] "numeric"
```

```
class(number2)
```

```
[1] "integer"
```

```
# use is.integer to check data type
is.integer(number1)
```

```
[1] FALSE
```

```
is.integer(number2)
```

```
[1] TRUE
```

Character

Letters, words and group of words are represented by the data type character. All data of type character must be enclosed in single or double quotation marks. In fact any value enclosed in quotes will be treated as character. Let us create two variables to store the first and last name of some random guy.

```
# first name
first_name <- "jovial"
```

```
# last name
last_name <- 'mann'
```

```
# check data type
class(first_name)
```

```
[1] "character"
```

```
class(last_name)
```

```
[1] "character"
```

```
# use is.character to check data type
is.character(first_name)
```

```
[1] TRUE
```

```
is.character(last_name)
```

```
[1] TRUE
```

You can coerce any data type to character using `as.character()`.

```
# create variable of different data types
age <- as.integer(30) # integer
score <- 9.8          # numeric/double
opt_course <- TRUE    # logical
today <- Sys.time()   # date time
```

```
as.character(age)
```

```
[1] "30"
```

```
as.character(score)
```

```
[1] "9.8"
```

```
as.character(opt_course)
```

```
[1] "TRUE"
```

```
as.character(today)
```

```
[1] "2026-10-02 22:53:29.976348"
```

Logical

Logical data types take only 2 values. Either TRUE or FALSE (plus NA for missing values, giving three-valued logic in R). Such data types are created when we compare two objects in R using comparison or logical operators.

- create two variables x and y
- assign them the values TRUE and FALSE respectively
- use `is.logical` to check data type
- use `as.logical` to coerce other data types to logical

```
# create variables x and y
```

```
x <- TRUE
```

```
y <- FALSE
```

```
# check data type
```

```
class(x)
```

```
[1] "logical"
```

```
is.logical(y)
```

```
[1] TRUE
```

The outcome of comparison operators is always logical. In the below example, we compare two numbers to see the outcome.

```
# create two numeric variables
```

```
x <- 3
```

```
y <- 4
```

```
# compare x and y
```

```
x > y
```

```
[1] FALSE
```

```
x < y
```

```
[1] TRUE
```

```
# store the result
```

```
z <- x > y
```

```
class(z)
```

```
[1] "logical"
```

TRUE is represented by all numbers except 0. FALSE is represented only by 0 and no other numbers.

```
# TRUE and FALSE are represented by 1 and 0
```

```
as.logical(1)
```

```
[1] TRUE
```

```
as.logical(0)
```

```
[1] FALSE
```

```
# using numbers
```

```
as.numeric(TRUE)
```

```
[1] 1
```

```
as.numeric(FALSE)
```

```
[1] 0
```

```
# using different numbers
```

```
as.logical(c(-2, -1.5, -1, 0, 1, 2))
```

```
[1] TRUE TRUE TRUE FALSE TRUE TRUE
```

Use `as.logical()` to coerce other data types to logical.

```
# create variable of different data types
```

```
age <- as.integer(30) # integer
```

```
score <- 9.8 # numeric/double
```

```
opt_course <- TRUE # logical
```

```
today <- Sys.time() # date time
```

```
as.logical(age)
```

```
[1] TRUE
```

```
as.logical(score)
```

```
[1] TRUE
```

```
as.logical(opt_course)
```

```
[1] TRUE
```

```
as.logical(today)
```

```
[1] TRUE
```

Date

R has dedicated types for dates and date-times. Use `Date` for calendar dates and `POSIXct` for date-times:

```
# today's date
today_date <- Sys.Date()
today_date

[1] "2026-10-02"

class(today_date)

[1] "Date"

# create a date from a string
independence_day <- as.Date("2026-01-15")
independence_day

[1] "2026-01-15"

# pretty-print a date
format(independence_day, "%B %d, %Y")

[1] "January 15, 2026"
```

For date-times, use `as.POSIXct()` (or the `lubridate` package, see the appendix in Phase 3).

Getting Help in R

Introduction

In this chapter, we will learn about the different methods of getting help in R. Often we get stuck while doing some analysis as either we do not know the correct function to use or its syntax. It is important for anyone who is new to R to know the right place to look for help. There are two ways to look for help in R:

- built in help system
- online

In the first section, we will look at various ways to access the built in help system of R. In the second section, we will look at various online resources that can supplement the built in help system. Let us get started!

Online Resources

R Bloggers

R Bloggers aggregates blogs written in english from across the globe. This is the first place you want to visit if you want help with R, data analysis, visualization and machine learning. There are blogs on a wide range of topics and the latest content is delivered to your inbox if you subscribe. If you are a blogger yourself, share it with the R community by adding your blog to R Bloggers.

Stack Overflow

Stack Overflow is a great place to visit if you are having trouble with R code or packages. Chances are high that someone has already encountered the same or similar problem and you can use the answers given by R experts. In case you have encountered a new problem or issue, you can ask for help by providing a reproducible example of your analysis along with the R code. Use the `reprex` package to create reproducible examples.

Twitter

The R community is very active on Twitter/X and there are lot of R experts who are willing to help those who are new to R. Use the hashtag `#rstats` if you are asking for help or guidance on Twitter/X. Many community members are also active on Mastodon, Bluesky, and LinkedIn.

RStudio Community

RStudio Community is similar to Stack Overflow. You can ask questions related to RStudio, Shiny and tidyverse.

RStudio

RStudio has very good resources including cheatsheets, webinars and blogs.

Reddit

Reddit is another place where you can look for help. The discussions are moderated by R experts. There are subreddits for `Rstats`, `Rlanguage`, `Rstudio` and `Rshiny`.

R Weekly

Visit RWeekly to get regular updates about the R community. You can find information about new packages, blogs, conferences, workshops, tutorials and R jobs.

R User Groups

There are several R User Groups active across the globe. You can find the list [here](#). Join the local user group to meet, discuss and learn from other R enthusiasts and experts.

Data Helpers

Data Helpers is a list of data analysts, scientists and engineers willing to offer guidance put together by Angela Bassa. Visit the website to learn more about how you can approach for help and guidance. (Note: this site appears unmaintained as of 2026; treat links there with caution.)

Internal

In this section, we will look at the following functions:

- `help.start()`
- `help()`
- `?`
- `??`
- `help.search()`
- `demo()`
- `example()`

- `library()`
- `vignette()`
- `browseVignettes()`

help.start

The `help.start()` function opens the documentation page in your browser. Here you can find manuals, reference and other materials.

```
help.start()
```

help

Use `help()` to access the documentation of functions and data sets. `?` is a shortcut for `help()` and returns the same information.

```
help(plot)
?plot
```

help.search

`help.search()` will search all sources of documentation and return those that match the search string. `??` is a shortcut for `help.search()` and returns the same information.

```
help.search('regression')
??regression
```

demo

`demo` displays an interactive demonstration of certain topics provided in a R package. Typing `demo()` in the console will list the demos available in all the R packages installed.

```
demo()
demo(scoping)
```

example

`example()` displays examples of the specified topic if available.

```
example('mean')
```

```
mean> x <- c(0:10, 50)
```

```
mean> xm <- mean(x)
```

```
mean> c(xm, mean(x, trim = 0.10))
[1] 8.75 5.50
```

Package Documentation

library

Access the documentation of a package using `help()` inside `library()`. The package need not be installed for accessing the documentation.

```
library(help = 'ggplot2')
```

vignette

A vignette is a long form guide to a R package. You can access the vignettes available using `vignette()`. It will display a list of vignettes available in installed packages.

```
vignette()
```

To access a specific vignette from a package, specify the name of the vignette and the package.

```
vignette('dplyr', package = 'dplyr')
```

browseVignettes

`browseVignettes()` is another way to access the vignettes in installed packages. It will list the vignettes in each package along with links to the web page and R code.

```
browseVignettes()
```

RsiteSearch

`RsiteSearch()` will search for specified topics in help pages, vignettes and task views using the search engine at this link and return the result in a browser.

```
RSiteSearch('glm')
```

Vectors in R

Introduction

Vector is the most basic data structure in R. It is a sequence of elements of the same data type. If the elements are of different data types, they will be coerced to a common type that can accommodate all the elements. Vectors are generally created using `c()` (concatenate function), although depending on the data type of vector being created, other methods can be used.

Numeric Vector

We will create a numeric vector using `c()` but you can use any function that creates a sequence of numbers. After that we will use `is.vector()` to check if it is a vector and `class` to check the data type.

```
# create a numeric vector
num_vect <- c(1, 2, 3)
```

```
# display the vector
num_vect
```

```
[1] 1 2 3
```

```
# check if it is a vector
is.vector(num_vect)
```

```
[1] TRUE
```

```
# check data type
class(num_vect)
```

```
[1] "numeric"
```

Let us look at other ways to create a sequence of numbers. We leave it as an exercise to the reader to understand the functions using help.

```
# using colon
```

```
vect1 <- 1:10
```

```
vect1
```

```
[1] 1 2 3 4 5 6 7 8 9 10
```

```
# using rep
```

```
vect2 <- rep(1, 5)
```

```
vect2
```

```
[1] 1 1 1 1 1
```

```
# using seq
```

```
vect3 <- seq(10)
```

```
vect3
```

```
[1] 1 2 3 4 5 6 7 8 9 10
```

Integer Vector

Creating an integer vector is similar to numeric vector except that we need to instruct R to treat the data as integer and not numeric or double. We will use the same methods we used for creating numeric vectors. To specify that the data is of type integer, we suffix the number with L.

```
# integer vector
```

```
int_vect <- c(1L, 2L, 3L)
```

```
int_vect
```

```
[1] 1 2 3
```

```
# check data type
```

```
class(int_vect)
```

```
[1] "integer"
```

```
# using colon
```

```
vect1 <- 1L:10L
```

```
vect1
```

```
[1] 1 2 3 4 5 6 7 8 9 10
```

```
# using rep
```

```
vect2 <- rep(1L, 5)
```

```
vect2
```

```
[1] 1 1 1 1 1
```

```
# using seq
vect3 <- seq(10L)
vect3

[1] 1 2 3 4 5 6 7 8 9 10
```

Character Vector

A character vector may contain a single character, a word or a group of words. The elements must be enclosed in single or double quotations.

```
# character vector
greetings <- c("hello", "good morning")
greetings

[1] "hello"          "good morning"

# check data type
class(greetings)

[1] "character"
```

Logical Vector

A vector of logical values will either contain TRUE or FALSE or both.

```
# logical vector
vect_logic <- c(TRUE, FALSE, TRUE, TRUE, FALSE)
vect_logic

[1] TRUE FALSE TRUE TRUE FALSE

# check data type
class(vect_logic)

[1] "logical"
```

In fact, you can create an integer vector and coerce it to type logical.

```
# integer vector
int_vect <- rep(0L:1L, 3)
int_vect

[1] 0 1 0 1 0 1

# coerce to logical vector
log_vect <- as.logical(int_vect)
log_vect

[1] FALSE TRUE FALSE TRUE FALSE TRUE

# check data type
class(log_vect)

[1] "logical"
```

Naming Vector Elements

It is possible to name the different elements of a vector. The advantage of naming vector elements is that we can later on use these names to access the elements. Use `names()` to specify the names of a vector. You can specify the names while creating the vector or add them later.

Method 1: Create vector and add names later

```
# create vector and add names later
vect1 <- c(1, 2, 3)

# name the elements of the vector
names(vect1) <- c("One", "Two", "Three")

# call vect1
vect1
##      One      Two      Three
##       1       2       3
```

Method 2: Specify names while creating vector

```
# specify names while creating vector
vect2 <- c(John = 1, Jack = 2, Jill = 3, Jovial = 4)

# call vect2
vect2
##      John      Jack      Jill      Jovial
##       1       2       3       4
```

Vector Coercion

Vectors are homogeneous i.e. all the elements of the vector must be of the same type. If we try to create a vector by combining different data types, the elements will be coerced to the most flexible type. The below table shows the order in which coercion occurs.

character data type is the most flexible while logical data type is the least flexible. If you try to combine any other data type with character, all the elements will be coerced to type character. In the absence of character data, all elements will be coerced to numeric. Finally, if the data does not include character or numeric types, all the elements will be coerced to integer type.

Case 1: Different Data Types

```
# vector of different data types
vect1 <- c(1, 1L, 'one', TRUE)

# call vect1
vect1
## [1] "1"      "1"      "one"    "TRUE"

# check data type
class(vect1)
## [1] "character"
```

Case 2: Numeric, Integer and Logical

```
# vector of different data types
vect1 <- c(1, 1L, TRUE)

# call vect1
vect1
## [1] 1 1 1

# check data type
class(vect1)
## [1] "numeric"
```

Case 3: Integer and Logical

```
# vector of different data types
vect1 <- c(1L, TRUE)

# call vect1
vect1
## [1] 1 1

# check data type
class(vect1)
## [1] "integer"
```

To summarize, below is the order in which coercion takes place:

logical → integer → double/numeric → character

R coerces toward the **most flexible** type. If a vector contains a character element, everything becomes character. Without character, elements coerce to numeric. Without numeric, elements coerce to integer.

Vector Operations

In this section, we look at simple operations that can be performed on vectors in R. Remember that the nature of the operations depends upon the type of data. Below are a few examples:

Case 1: Vectors of same length

```
# create two vectors
vect1 <- c(1, 3, 8, 4)
vect2 <- c(2, 7, 1, 9)

# addition
vect1 + vect2
## [1] 3 10 9 13

# subtraction
vect1 - vect2
## [1] -1 -4 7 -5
```

```
# multiplication
vect1 * vect2
## [1] 2 21 8 36

# division
vect1 / vect2
## [1] 0.5000000 0.4285714 8.0000000 0.4444444
```

Case 2: Vectors of different length

In the previous case, the length i.e. the number of elements in the vectors were same. What happens if the length of the vectors are unequal? In such cases, the shorter vector is recycled to match the length of the longer vector. The below example should clear this concept:

```
# create two vectors
vect1 <- c(2, 7)
vect2 <- c(1, 8, 5, 2)

# addition
vect1 + vect2
## [1] 3 15 7 9

# subtraction
vect1 - vect2
## [1] 1 -1 -3 5

# multiplication
vect1 * vect2
## [1] 2 56 10 14

# division
vect1 / vect2
## [1] 2.000 0.875 0.400 3.500
```

Missing Data

Missing data is a reality. No matter how careful you are in collecting data for your analysis, chances are always high that you end up with some missing data. In R missing values are represented by NA. In this section, we will focus on the following:

- test for missing data
- remove missing data
- exclude missing data from analysis

Detect missing data

We first create a vector with missing values. After that, we will use `is.na()` to test whether the data contains missing values. `is.na()` returns a logical vector equal to the length of the vector being tested. Another function that can be used for detecting missing values is `complete.cases()`. Below is an example:

```
# vector with missing values
vect1 <- c(1, 3, NA, 5, 2)

# use is.na
is.na(vect1)
## [1] FALSE FALSE TRUE FALSE FALSE

# use complete.cases
complete.cases(vect1)
## [1] TRUE TRUE FALSE TRUE TRUE
```

Omit missing data

In the presence of missing data, all computations in R will return NA. To avoid this, we might want to remove the missing data before doing any computation. `na.omit()` will remove all missing values from the data. Let us look at an example:

```
# vector with missing values
vect1 <- c(1, 3, NA, 5, 2)

# call vect1
vect1
## [1] 1 3 NA 5 2

# omit missing values
na.omit(vect1)
## [1] 1 3 5 2
## attr("na.action")
## [1] 3
## attr("class")
## [1] "omit"
```

Exclude missing data

To exclude missing values from computations, use `na.rm` and set it to TRUE.

```
# vector with missing values
vect1 <- c(1, 3, NA, 5, 2)

# compute mean
mean(vect1)
## [1] NA

# compute mean by excluding missing value
mean(vect1, na.rm = TRUE)
## [1] 2.75
```

Index Vectors

One of the most important steps in data analysis is selecting a subset of data from a bigger data set. Indexing helps in retrieving values individually or a set of values that meet a specific criteria. In this chapter, we look at various ways of indexing/subsetting vectors.

[] is the index operator in R. We can use various expressions within [] to subset data. In R, **index positions begin at 1 and not 0**. To begin with, let us look at values in different index positions:

```
# fixed seed so this chapter re-renders deterministically
set.seed(42)
```

```
# random sample of 10 values
vect1 <- sample(10)
vect1

[1] 1 5 10 8 2 4 6 9 7 3
```

```
# return third element
vect1[3]
```

```
[1] 10
```

```
# return seventh element
vect1[7]
```

```
[1] 6
```

Out of range index

```
# random sample of 10 values
vect1 <- sample(10)
vect1

[1] 8 7 4 1 5 10 2 6 9 3
```

```
# return value at index 0
vect1[0]
```

```
integer(0)
```

```
# length of the vector
length(vect1)
```

```
[1] 10
```

```
# out of range index
vect1[11]
```

```
[1] NA
```

In the first case, we specified the index as 0 and in the second case we used the index 11, which is greater than the length of the vector. R returns an empty vector in the first case and NA in the second case.

Negative Index

Using a negative index will delete the value in the said index position. Unlike other languages, it will not index elements from the end of the vector counting backwards. Let us look at an example to understand how negative index works in R:

```
# random sample of 10 values
vect1 <- sample(10)
vect1

[1] 9 10 3 4 5 6 1 2 8 7

# drop third element
vect1[-3]

[1] 9 10 4 5 6 1 2 8 7

# drop seventh element
vect1[-7]

[1] 9 10 3 4 5 6 2 8 7
```

Subset Multiple Elements

If we do not specify anything within [], all the elements in the vector will be returned. We can specify the index elements using any expression that generates a sequence of integers. Let us look at a few examples:

```
# random sample of 10 values
vect1 <- sample(10)
vect1

[1] 3 1 2 6 10 8 4 5 7 9

# return all elements
vect1[]

[1] 3 1 2 6 10 8 4 5 7 9

# return first 5 values
vect1[1:5]

[1] 3 1 2 6 10

# return all values from the 5th position
end <- length(vect1)
vect1[5:end]

[1] 10 8 4 5 7 9
```

If you are using the colon to generate the index positions, you will have to specify both the starting and ending position, else, R will return an error.

What if we want elements that are not in a sequence as we saw in the last example? In such cases, we have to create a vector using `c()` and use it to extract elements from the original vector. Below is an example:

```
# random sample of 10 values
vect1 <- sample(10)
vect1

[1] 5 4 2 8 3 1 10 7 6 9
```

```
# extract 2nd, 5th and 7th element
select <- c(2, 5, 7)
vect1[select]

[1] 4 3 10

# extract elements in position 1 to 4, 6 and 9
select <- c(1:4, 6, 9)
vect1[select]

[1] 5 4 2 8 1 6
```

Subset Named Vectors

Vectors can be subset using the name of the elements. **When using name of elements for subsetting, ensure that the names are enclosed in single or double quotations**, else R will return an error. Let us look at a few examples:

```
vect1 <- c(score1 = 8, score2 = 6, score3 = 9)
vect1

score1 score2 score3
      8      6      9

# extract score2
vect1['score2']

score2
      6

# extract score1 and score3
vect1[c('score1', 'score3')]

score1 score3
      8      9
```

Subset using logical values

Logical values can be used to subset vectors. They are not very flexible but can be used for simple indexing. In all of the below examples, the logical vectors are recycled to match the length of the vector from which we subset data:

```
# random sample of 10 values
vect1 <- sample(10)
vect1

[1] 6 2 4 3 10 5 9 7 8 1

# returns all values
vect1[TRUE]

[1] 6 2 4 3 10 5 9 7 8 1

# empty vector
vect1[FALSE]
```

```
integer(0)

# values in odd positions
vect1[c(TRUE, FALSE)]

[1] 6 4 10 9 8

# values in even positions
vect1[c(FALSE, TRUE)]

[1] 2 3 5 7 1
```

Subset using logical expressions

Logical expressions can be used to extract elements that meet specific criteria. This method is most flexible and useful as we can combine multiple conditions using relational and logical operators. Before we use logical expressions, let us spend some time understanding comparison and logical operators as we will be using them extensively hereafter.

Comparison Operators

When you create an expression using a comparison operator, the output is always a logical value i.e. TRUE or FALSE. Let us see how we can use comparison operators to subset data:

```
# random sample of 10 values
vect1 <- sample(10)
vect1

[1] 10 8 5 1 7 4 6 2 9 3

# return elements greater than 5
vect1 > 5

[1] TRUE TRUE FALSE FALSE TRUE FALSE TRUE FALSE TRUE FALSE

vect1[vect1 > 5]

[1] 10 8 7 6 9

# return elements greater than or equal to 5
vect1 >= 5

[1] TRUE TRUE TRUE FALSE TRUE FALSE TRUE FALSE TRUE FALSE

vect1[vect1 >= 5]

[1] 10 8 5 7 6 9

# return elements lesser than 5
vect1 < 5

[1] FALSE FALSE FALSE TRUE FALSE TRUE FALSE TRUE FALSE TRUE

vect1[vect1 < 5]

[1] 1 4 2 3
```

```

# return elements lesser than or equal to 5
vect1 <= 5

[1] FALSE FALSE TRUE TRUE FALSE TRUE FALSE TRUE FALSE TRUE

vect1[vect1 <= 5]

[1] 5 1 4 2 3

# return elements equal to 5
vect1 == 5

[1] FALSE FALSE TRUE FALSE FALSE FALSE FALSE FALSE FALSE

vect1[vect1 == 5]

[1] 5

# return elements not equal to 5
vect1 != 5

[1] TRUE TRUE FALSE TRUE TRUE TRUE TRUE TRUE TRUE TRUE

vect1[vect1 != 5]

[1] 10 8 1 7 4 6 2 9 3

```

Logical Operators

Let us combine comparison and logical operators to create expressions and use them to subset vectors:

```

# random sample of 10 values
vect1 <- sample(10)
vect1

[1] 4 9 3 8 5 6 10 2 1 7

# return all elements less than 8 or divisible by 3
vect1[(vect1 < 8 | (vect1 %% 3 == 0))]

[1] 4 9 3 5 6 2 1 7

# return all elements less than 7 or divisible by 2
vect1[(vect1 < 7 | (vect1 %% 2 == 0))]

[1] 4 3 8 5 6 10 2 1

```

Dataframes in R

Introduction

In this chapter, we will learn to:

- create dataframe
- select columns
- select rows

- utility functions

Create dataframes

Use `data.frame` to create dataframes. Below is the function syntax:

```
args(data.frame)

function (... , row.names = NULL, check.rows = FALSE, check.names = TRUE,
        fix.empty.names = TRUE, stringsAsFactors = FALSE)
NULL
```

Data frames are basically lists with elements of equal length and as such, they are heterogeneous.

Let us create a dataframe:

```
name <- c('John', 'Jack', 'Jill')
age <- c(29, 25, 27)
graduate <- c(TRUE, TRUE, FALSE)
students <- data.frame(name, age, graduate)
students
```

```
  name age graduate
1 John  29      TRUE
2 Jack  25      TRUE
3 Jill  27     FALSE
```

Basic Information

```
class(students)
## [1] "data.frame"
names(students)
## [1] "name"      "age"       "graduate"
colnames(students)
## [1] "name"      "age"       "graduate"
str(students)
## 'data.frame':  3 obs. of  3 variables:
## $ name      : chr  "John" "Jack" "Jill"
## $ age       : num  29 25 27
## $ graduate: logi  TRUE TRUE FALSE
dim(students)
## [1] 3 3
nrow(students)
## [1] 3
ncol(students)
## [1] 3
```

Select Columns

Single Column

- `[]`
- `[[ ]]`
- `$`

```
# using [
students[1]
##   name
## 1 John
## 2 Jack
## 3 Jill

# using [[
students[[1]]
## [1] "John" "Jack" "Jill"

# using $
students$name
## [1] "John" "Jack" "Jill"
```

Multiple Columns

```
students[, 1:3]
##   name age graduate
## 1 John  29      TRUE
## 2 Jack  25      TRUE
## 3 Jill  27     FALSE

students[, c(1, 3)]
##   name graduate
## 1 John      TRUE
## 2 Jack      TRUE
## 3 Jill     FALSE
```

Select Rows

```
# single row
students[1, ]
##   name age graduate
## 1 John  29      TRUE

# multiple row
students[c(1, 3), ]
##   name age graduate
## 1 John  29      TRUE
## 3 Jill  27     FALSE
```

Starting with R 4.0.0 (April 2020), the default value of `stringsAsFactors` in `data.frame()` is `FALSE`, meaning character columns remain as character type. In older versions of R (`< 4.0.0`), the default was `TRUE`, which automatically converted character vectors to factors.

```
students <- data.frame(name, age, graduate, stringsAsFactors = FALSE)
```

Factors in R

Introduction

In this chapter, we will learn to:

- create factors
- order levels
- specify labels
- check levels
- number of levels

Categorical or qualitative data in R is treated as data type factor.

Create Factors

```
args(factor)
```

```
function (x = character(), levels, labels = levels, exclude = NA,  
         ordered = is.ordered(x), nmax = NA)
```

```
NULL
```

```
devices <- factor(c('Mobile', 'Tablet', 'Desktop'))
```

```
devices
```

```
[1] Mobile Tablet Desktop
```

```
Levels: Desktop Mobile Tablet
```

```
# number of levels
```

```
nlevels(devices)
```

```
[1] 3
```

```
# levels
```

```
levels(devices)
```

```
[1] "Desktop" "Mobile"  "Tablet"
```

Order Levels

Use the levels argument together with ordered = TRUE when the categories have a natural order:

```
size <- factor(c('Medium', 'Low', 'High', 'Low', 'Medium'),
```

```
             levels = c('Low', 'Medium', 'High'), ordered = TRUE)
```

```
size
```

```
[1] Medium Low    High   Low    Medium
```

```
Levels: Low < Medium < High
```

```
levels(size)
```

```
[1] "Low"    "Medium" "High"
```

Specify Labels

Use the `labels` argument to attach shorter or prettier display labels while keeping the underlying level order:

```
size_labeled <- factor(c('Low', 'Medium', 'High'),
  levels = c('Low', 'Medium', 'High'), labels = c('L', 'M', 'H'))
size_labeled

[1] L M H
Levels: L M H
```

Check Levels

```
# existing levels
levels(size)

[1] "Low"      "Medium"   "High"

# number of levels
nlevels(size)

[1] 3

# is this an ordered factor?
is.ordered(size)

[1] TRUE
```

For modern factor manipulation (reordering, recoding, lumping rare levels), see the `forcats` package.

Lists in R

Introduction

Lists are very useful as they are heterogeneous i.e. they can contain different data types. If you remember, vectors and matrices are homogeneous i.e. they can contain only one type of data. If you include different data types, they will all be coerced to the same type. With lists, it is different. In this chapter, we will delve deeper into lists.

- how to create lists
- access list elements
- name list elements
- coerce other R objects to list
- coerce list to other R objects

Creating Lists

To create a list, we use the `list()` function. Let us create a simple list to demonstrate how they can contain different data types.

```
# numeric vector
vect1 <- 1:10
```

```

# character vector
vect2 <- c('Jack', 'John', 'Jill')

# logical vector
vect3 <- c(TRUE, FALSE)

# matrix
mat <- matrix(data = 1:9, nrow = 3)

# list
l <- list(vect1, vect2, vect3, mat)
l

[[1]]
[1] 1 2 3 4 5 6 7 8 9 10

[[2]]
[1] "Jack" "John" "Jill"

[[3]]
[1] TRUE FALSE

[[4]]
      [,1] [,2] [,3]
[1,]    1    4    7
[2,]    2    5    8
[3,]    3    6    9

```

If you observe the output, all the elements of the list retain their data types. Now let us learn how to access the elements of the list.

Access List Elements

You can access the elements of a list using the following operators:

- `[[`
- `[`
- `$`

Let us try them one by one.

```

# using [[
l[[1]]

[1] 1 2 3 4 5 6 7 8 9 10

# using [
l[1]

[[1]]
[1] 1 2 3 4 5 6 7 8 9 10

```

`[]` extracts the element itself (which may be a vector, matrix, data frame, or nested list), while `[]` returns a new list containing the selected element(s). The `$` operator can be used only when we have named elements in the list. Let us add names to the elements. Use the `names()` function to add names to the list.

```
# named elements
names(l) <- c('vect1', 'vect2', 'vect3', 'mat')
l

$vect1
[1] 1 2 3 4 5 6 7 8 9 10

$vect2
[1] "Jack" "John" "Jill"

$vect3
[1] TRUE FALSE

$mat
      [,1] [,2] [,3]
[1,]    1    4    7
[2,]    2    5    8
[3,]    3    6    9

# use $
l$vect1

[1] 1 2 3 4 5 6 7 8 9 10

# use []
l[['vect1']]

[1] 1 2 3 4 5 6 7 8 9 10

# use [
l['vect1']

$vect1
[1] 1 2 3 4 5 6 7 8 9 10
```

Coerce other objects

You can coerce other objects to list using `as.list()`.

```
# vector
vect1 <- 1:10
as.list(vect1)

[[1]]
[1] 1

[[2]]
[1] 2
```

```
[[3]]  
[1] 3
```

```
[[4]]  
[1] 4
```

```
[[5]]  
[1] 5
```

```
[[6]]  
[1] 6
```

```
[[7]]  
[1] 7
```

```
[[8]]  
[1] 8
```

```
[[9]]  
[1] 9
```

```
[[10]]  
[1] 10
```

```
# matrix
```

```
mat <- matrix(data = 1:9, nrow = 3)  
as.list(mat)
```

```
[[1]]  
[1] 1
```

```
[[2]]  
[1] 2
```

```
[[3]]  
[1] 3
```

```
[[4]]  
[1] 4
```

```
[[5]]  
[1] 5
```

```
[[6]]  
[1] 6
```

```
[[7]]  
[1] 7
```

```
[[8]]
```

```
[1] 8
```

```
[[9]]  
[1] 9
```

Coerce list to other objects

Use `unlist()` to coerce a list to a vector.

```
# unlist  
unlist(l)
```

vect11	vect12	vect13	vect14	vect15	vect16	vect17	vect18	vect19	vect110
"1"	"2"	"3"	"4"	"5"	"6"	"7"	"8"	"9"	"10"
vect21	vect22	vect23	vect31	vect32	mat1	mat2	mat3	mat4	mat5
"Jack"	"John"	"Jill"	"TRUE"	"FALSE"	"1"	"2"	"3"	"4"	"5"
mat6	mat7	mat8	mat9						
"6"	"7"	"8"	"9"						

```
# remove names  
unlist(l, use.names = FALSE)
```

[1]	"1"	"2"	"3"	"4"	"5"	"6"	"7"	"8"	"9"
[10]	"10"	"Jack"	"John"	"Jill"	"TRUE"	"FALSE"	"1"	"2"	"3"
[19]	"4"	"5"	"6"	"7"	"8"	"9"			

Install & Update R Packages

Introduction

In this chapter, we will learn about R packages. Packages are fundamental to R. There are more than 22,000 packages available on CRAN or the Comprehensive R Archive Network (as of September 2026; verify the current count at cran.r-project.org/web/packages).

Packages are available for different topics. You should always look for a package before writing code from scratch. In case you have written your own codes for a new analysis or topic, do share it with the R community by converting the code into a package. You can learn more about building R packages from *R Packages*, a book written by Hadley Wickham.

In this chapter, we will learn to:

- install R packages from
 - CRAN
 - GitHub
 - BitBucket
 - Bioconductor
 - rForge
- install different versions of a package
- load, update & remove installed packages
- access package documentation

Install Packages

CRAN

Packages from CRAN can be installed using `install.packages()`. The name of the package must be enclosed in single or double quotes.

```
install.packages('ggplot2')
```

GitHub

Some R packages are made available on GitHub before releasing them on CRAN. Such packages can be installed using `install_github()` from `devtools` or `remotes` package. You need to specify the name of the repository and the package. For example, to download `ggplot2` or `dplyr`, below is the code:

```
devtools::install_github("tidyverse/ggplot2")
remotes::install_github("tidyverse/dplyr")
```

BitBucket

Bitbucket is similar to GitHub. You can install packages from Bitbucket using `install_bitbucket()` from `devtools` or `remotes` package.

```
devtools::install_bitbucket("dannavarro/lsr-package")
remotes::install_bitbucket("dannavarro/lsr-package")
```

Bioconductor

Bioconductor provides tools for analysis and comprehension of high throughput genomic data. Install the `BiocManager` package and use `BiocManager::install()`:

```
# install BiocManager if needed
install.packages("BiocManager")

# install a Bioconductor package
BiocManager::install("GenomicFeatures")
```

rForge

Many R packages are hosted at R-Forge, a platform for development of R packages. (Note: R-Forge is largely inactive today; prefer CRAN or GitHub for new work.)

```
install.packages('quantstrat', repos = 'https://r-forge.r-project.org/')
```

For faster installs with dependency resolution, you can also try `pak::pkg_install()` from the `pak` package as a modern alternative.

Install Different Versions

Now that we have learnt how to install packages, let us look at installing different versions of the same package.

```
remotes::install_version('dplyr', version = 0.5.0)
```

If you want to install the latest release from GitHub, append `@*release` to the repository name. For example, to install the latest release of `dplyr`:

```
remotes::install_github('tidyverse/dplyr@*release')
```

Installed Packages

- `installed.packages()`: view currently installed packages
- `library('package_name')`: load packages
- `autoload('function_name', 'package_name')`: load functions and data from packages only when called in the script
- `available.packages()`: packages available for installation
- `old.packages()`: packages which have new versions available
- `new.packages()`: packages already not installed
- `update.packages()`: update packages which have new versions
- `remove.packages('package_name')`: remove installed packages

Library Paths

Library is a directory that contains all installed packages. Usually there will be more than one R library in your system. You can find the location of the libraries using `.libPaths()`.

```
.libPaths()
```

```
[1] "C:/Users/HP/AppData/Local/R/cache/R/renv/library/intro-r-15ca35ae/windows/R-4.5/x86_64-w64-mingw32"
[2] "C:/Users/HP/AppData/Local/R/cache/R/renv/sandbox/windows/R-4.5/x86_64-w64-mingw32/ebc25411"
```

You can use `lib` when you want to install, load, update and remove packages from a particular library.

Install

```
install.packages('stringr', lib = .libPaths()[1])
```

Load

```
library(lubridate, lib = .libPaths()[1])
```

Update Packages

```
update.packages(lib = .libPaths()[1])
```

Remove Packages

```
remove.packages('stringr', lib = .libPaths()[1])
```

Explore Rsquared Academy's own R packages and guides at pkgs.rsquaredacademy.com.

RStudio Projects & Data Import

Learning Objectives: After this chapter you will be able to: - Work inside an RStudio Project (.Rproj) instead of `setwd()`. - Build portable paths with `here::here()`. - Read CSV files with `readr::read_csv()` and Excel files with `readxl::read_excel()`.

```
library(readr)
library(readxl)
library(here)
```

Why avoid `setwd()`

`setwd("C:/Users/you/project")` hard-codes a path that exists only on your machine. Anyone else who opens the script — or you on a different computer — gets an error. An RStudio Project (.Rproj) fixes this: opening the .Rproj file sets the working directory to the project folder automatically, so relative paths just work.

Workflow: File → New Project → New Directory (or Existing Directory) → give it a name. From then on, open the .Rproj file, not individual scripts.

Portable paths with `here`

The `here` package builds paths from the project root, wherever the project lives on disk:

```
here("data", "students.csv")

[1] "F:/R/ebooks/intro-r/data/students.csv"
```

Common Mistake: Paths with `../` or `C:/...` break when the project moves. If a path starts with a drive letter or `../`, rewrite it with `here()`.

Reading CSV files

Practice data lives in the `data/` directory (CC0 / public domain, created for this book). Read it with `read_csv()`:

```
students <- read_csv(here("data", "students.csv"))
students

# A tibble: 5 x 3
  name    age score
  <chr> <dbl> <dbl>
1 Asha    21    88
2 Ravi    22    92
3 Meera   20    79
4 Arjun   23    95
5 Divya   21    84
```

Reading Excel files

`read_excel()` works the same way for `.xlsx` files — sheet 1 by default:

```
students_xl <- read_excel(here("data", "students.xlsx"))
students_xl
```

```
# A tibble: 5 x 3
  name    age score
  <chr> <dbl> <dbl>
1 Asha    21    88
2 Ravi    22    92
3 Meera   20    79
4 Arjun   23    95
5 Divya   21    84
```

Practice Check

Knowledge Check: *Project-relative paths* Why does `read_csv(here("data", "students.csv"))` keep working when you move the whole project folder to another computer, but `read_csv("C:/Users/you/project/data/students.csv")` does not?

Click to reveal answer **Answer:** `here()` resolves the path relative to the project root at run time, so it adapts to the new location. The `C:/...` path is absolute and points to a folder that does not exist on the other computer.

Explanation: Projects + `here()` keep every path portable (project root $\rightarrow$ data $\rightarrow$ file); absolute paths and `setwd()` tie the script to one machine. **Knowledge Check:** *Why .Rproj beats setwd()* What breaks when you share a script containing `setwd("C:/Users/you/project")` with a colleague?

Click to reveal answer **Answer:** The script errors because that folder does not exist on your colleague's machine.

Explanation: `setwd()` with an absolute path ties the script to one computer; opening a shared `.Rproj` file sets everyone's working directory to the project folder automatically. **Knowledge Check:** *here() resolution* Your project lives at `D:/work/survey` and contains `data/resp.csv`. What does `here("data", "resp.csv")` return?

Click to reveal answer **Answer:** `D:/work/survey/data/resp.csv` (i.e. project root $\rightarrow$ data $\rightarrow$ resp.csv).

Explanation: `here()` anchors paths at the project root (where the `.Rproj` file lives), joining each additional component with the correct separator for your OS. **Knowledge Check:** *read_csv() vs read.csv()* Name one practical advantage of `readr::read_csv()` over base R's `read.csv()` for a beginner.

Click to reveal answer **Answer:** Any one of: it never converts strings to factors, it prints a compact tibble preview, or it reports column types as it reads.

Explanation: `read_csv()` uses modern defaults (no `stringsAsFactors` surprises) and tells you the parsed type of each column, so mismatches are visible immediately. **Knowledge Check:** *Excel sheets* `read_excel(here("data", "students.xlsx"))` reads which sheet by default, and how would you read the second sheet instead?

Click to reveal answer **Answer:** The first sheet by default; pass `sheet = 2` (or the sheet name) to read the second sheet.

Explanation: `read_excel()` defaults to sheet 1; the `sheet` argument selects another sheet by position or name.

Your First Visualization

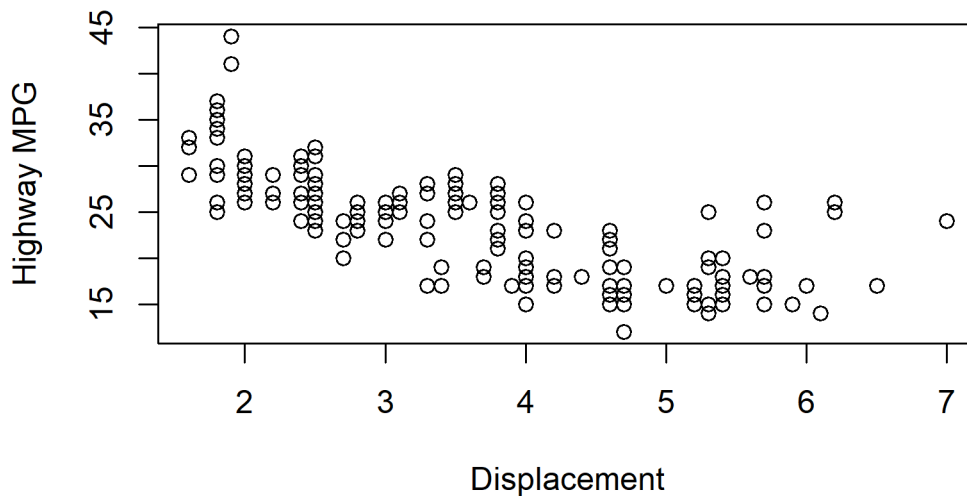
Learning Objectives: After this chapter you will be able to: - Load `ggplot2` and its bundled `mpg` dataset. - Draw the same scatterplot in base R and in `ggplot2`. - Describe a plot as data + aesthetics + geometry.

```
library(ggplot2)
# ggplot2 bundles the `mpg` dataset (MIT license, same as ggplot2 itself)
data(mpg, package = "ggplot2")
```

Common Mistake: Base R has no `mpg` dataset (it has `mtcars`). `plot(mpg$displ, mpg$hwy)` without `library(ggplot2)` fails with object '`mpg`' not found. Always load `ggplot2` first.

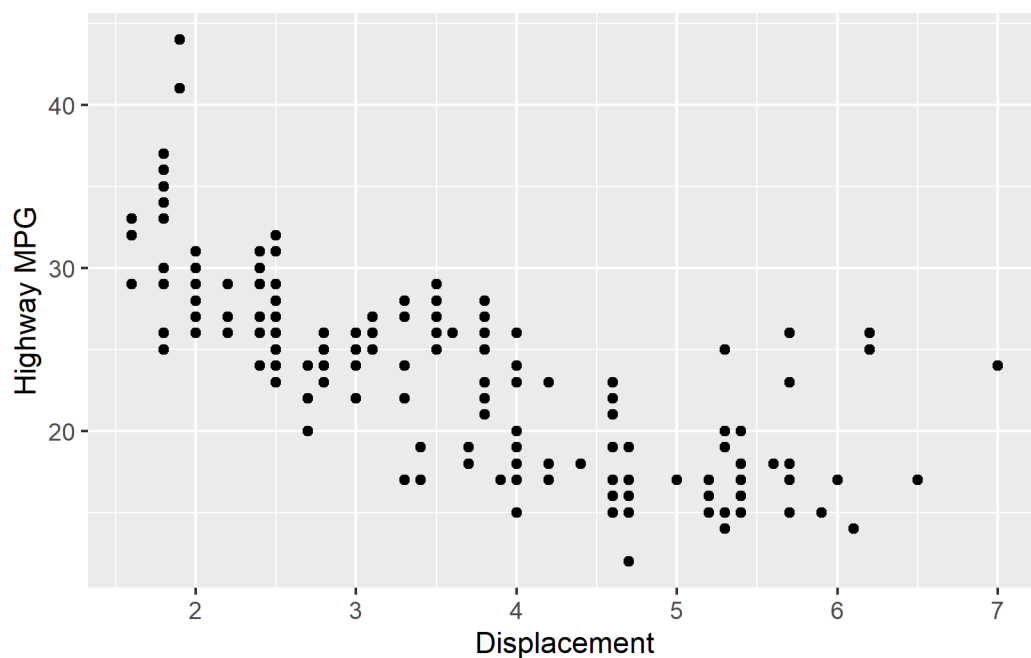
Base R graphics

```
plot(mpg$displ, mpg$hwy, xlab = "Displacement", ylab = "Highway MPG")
```



ggplot2 graphics

```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point() +
  labs(x = "Displacement", y = "Highway MPG")
```



Mental model: every ggplot is `data(mpg) + aesthetics(aes(x, y)) + geometry(geom_point())`. Change the geometry, keep the mapping.

Practice Check

Knowledge Check: *Data + aesthetics + geometry* In `ggplot(mpg, aes(x = displ, y = hwy)) + geom_point()`, which part is the data, which is the aesthetics mapping, and which is the geometry?

Click to reveal answer **Answer:** Data: `mpg`. Aesthetics: `aes(x = displ, y = hwy)`. Geometry: `geom_point()`.

Explanation: `ggplot()` takes the data and the variable mapping; each `geom_*()` layer decides how those mapped variables are drawn. **Knowledge Check:** *Why library(ggplot2) comes first* What error do you get from `plot(mpg$displ, mpg$hwy)` in a fresh R session, and why?

Click to reveal answer **Answer:** object 'mpg' not found — base R has no `mpg` dataset (it has `mtcars`); `mpg` only exists after `library(ggplot2)` (or `data(mpg, package = "ggplot2")`).

Explanation: Datasets live inside packages. Until the package is loaded, its data is invisible to your session. **Knowledge Check:** *Changing the geometry* How would you turn the scatterplot into a smoothed trend line plot without changing the data or the variable mapping?

Click to reveal answer **Answer:** Replace `geom_point()` with `geom_smooth()` (or add it as a second layer).

Explanation: Data + aesthetics stay fixed while geometries are interchangeable layers — that is the point of the grammar of graphics. **Knowledge Check:** *Aesthetics mapping* In `aes(x = displ, y = hwy)`, what do `x` and `y` refer to — plot labels or dataset columns?

Click to reveal answer **Answer:** Dataset columns: `mpg$displ` mapped to horizontal position, `mpg$hwy` to vertical position.

Explanation: `aes()` maps columns of the data to visual properties. Labels are set separately with `labs()`. **Knowledge Check:** *Base R vs ggplot2* The base R call repeats `mpg$` twice; the `ggplot2` call mentions `mpg` once. Why is that difference useful as plots get more complex?

Click to reveal answer **Answer:** Declaring the data once in `ggplot(mpg, ...)` means every layer inherits it — you never retype the data source per variable, so longer plots stay readable and less error-prone.

Explanation: One data declaration shared across layers beats repeating `data$column` for every variable, especially with facets and multiple geoms.

Your First Data Wrangling

Learning Objectives: After this chapter you will be able to: - Use the five core `dplyr` verbs: `filter()`, `select()`, `mutate()`, `arrange()`, `summarise()`. - Chain steps with the native base pipe `|>`. - Know where to go next after this book.

```
library(dplyr)
# ggplot2 bundles the `mpg` dataset (MIT license, same as ggplot2 itself)
data(mpg, package = "ggplot2")
```

We use the native base pipe `|>` (standard since R 4.1.0; zero external dependencies). We do **not** use `%>%`.

The five verbs

```
mpg |>
  filter(manufacturer == "audi") |>
  select(manufacturer, model, displ, hwy) |>
  mutate(km_per_litre = hwy * 0.425144) |>
  arrange(desc(hwy))
```

A tibble: 18 x 5

	manufacturer	model	displ	hwy	km_per_litre
	<chr>	<chr>	<dbl>	<int>	<dbl>
1	audi	a4	2	31	13.2
2	audi	a4	2	30	12.8
3	audi	a4	1.8	29	12.3
4	audi	a4	1.8	29	12.3
5	audi	a4 quattro	2	28	11.9
6	audi	a4	3.1	27	11.5
7	audi	a4 quattro	2	27	11.5
8	audi	a4	2.8	26	11.1
9	audi	a4	2.8	26	11.1
10	audi	a4 quattro	1.8	26	11.1
11	audi	a4 quattro	1.8	25	10.6

```

12 audi          a4 quattro  2.8    25      10.6
13 audi          a4 quattro  2.8    25      10.6
14 audi          a4 quattro  3.1    25      10.6
15 audi          a4 quattro  3.1    25      10.6
16 audi          a6 quattro  3.1    25      10.6
17 audi          a6 quattro  2.8    24      10.2
18 audi          a6 quattro  4.2    23       9.78

```

```

mpg |>
  summarise(mean_hwy = mean(hwy), n = dplyr::n())

# A tibble: 1 x 2
  mean_hwy     n
    <dbl> <int>
1    23.4   234

```

Common Mistake: `mean()` on a vector with NA returns NA. Add `na.rm = TRUE` when missing values are present.

Practice Check

Knowledge Check: *Filter then select* What does `mpg |> filter(manufacturer == "audi") |> select(model, hwy)` return?

Click to reveal answer **Answer:** A data frame with two columns (`model`, `hwy`) containing only the rows where `manufacturer` is "audi".

Explanation: `filter()` keeps matching rows; `select()` keeps matching columns. The `|>` pipe passes the left-hand result into the first argument of the right-hand function. **Knowledge Check:** *mutate() adds columns* What does `mpg |> mutate(km_per_litre = hwy * 0.425144)` return, and how many rows does the result have compared to `mpg`?

Click to reveal answer **Answer:** The full `mpg` data with one extra column (`km_per_litre`); the row count is unchanged.

Explanation: `mutate()` adds (or modifies) columns row-wise — it never adds or removes rows. Use `filter()` to change row counts. **Knowledge Check:** *arrange() ordering* After `mpg |> arrange(desc(hwy))`, which kind of car sits in row 1?

Click to reveal answer **Answer:** The car with the highest highway mileage (`hwy`).

Explanation: `arrange()` sorts ascending by default; wrapping in `desc()` flips it to descending, so the maximum comes first. **Knowledge Check:** *summarise() collapses rows* How many rows does `mpg |> summarise(mean_hwy = mean(hwy), n = dplyr::n())` return, and why?

Click to reveal answer **Answer:** Exactly one row: the mean of `hwy` across the whole dataset and the total row count.

Explanation: Without `group_by()`, `summarise()` collapses the entire data frame into a single summary row. **Knowledge Check:** *Reading a pipeline* Read this pipeline aloud in order: `mpg |`

```
> filter(manufacturer == "audi") |> select(model, hwy) |> arrange(desc(hwy)).
```

 What question does it answer?

Click to reveal answer **Answer:** “Among Audi models, which has the best (and worst) highway mileage?” — it keeps only Audis, keeps the model and mileage columns, and sorts best-first.

Explanation: Pipelines read top-to-bottom as a sequence of verbs; narrating each step in plain words is the fastest way to check you built the right one.

Where to go next

Where to go next: - Book: R for Data Science (2e) — start at Chapter 1 - Courses: Rsquared Academy Data Wrangling series - More books: ebooks.rsquaredacademy.com